

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)

ARTIFICIAL INTELLIGENCE (AI)

Practical – VII

Roll No.: T003	Name: Pranay Bhadwalkar
Class: TYIT	Batch: B1
Date of Assignment: 01/08/2026	Date/Time of Submission: 01/08/2026

1. Using any small dataset (e.g., Iris or a CSV dataset), write a Python program to demonstrate the basic machine learning workflow.

iris.csv

a. Load dataset using Pandas.

```
% Generate + Code + Markdown

# Import libraries
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

[38] ✓ 0.0s Python

a. Load dataset using Pandas

# Load Dataset
df = pd.read_csv("iris.csv")

[39] ✓ 0.0s Python

# Display Dataset
print(df.head())

[40] ✓ 0.0s Python

...
  sepal_length  sepal_width  petal_length  petal_width  species
0           5.1           3.5           1.4           0.2  setosa
1           4.9           3.0           1.4           0.2  setosa
2           4.7           3.2           1.3           0.2  setosa
3           4.6           3.1           1.5           0.2  setosa
4           5.0           3.6           1.4           0.2  setosa

# Info
print(df.info())

[41] ✓ 0.0s Python

...
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype  
---  --
 0   sepal_length  150 non-null     float64
 1   sepal_width   150 non-null     float64
 2   petal_length  150 non-null     float64
 3   petal_width   150 non-null     float64
 4   species       150 non-null     str
dtypes: float64(4), str(1)
memory usage: 6.0 KB
None

# Shape
print(df.shape)

[42] ✓ 0.0s Python

...
(150, 5)

# Columns
print(df.columns)

[43] ✓ 0.0s Python

...
Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
      'species'],
      dtype='str')
```

```
# Summary
print(df.describe())

(44) ✓ 0.0s Python

...
count    150.000000    150.000000    150.000000    150.000000
mean      5.843333     3.054000     3.758667     1.198667
std       0.828066     0.433594     1.764420     0.763161
min       4.300000     2.000000     1.000000     0.100000
25%       5.100000     2.800000     1.600000     0.300000
50%       5.800000     3.000000     4.350000     1.300000
75%       6.400000     3.300000     5.100000     1.800000
max       7.900000     4.400000     6.900000     2.500000

# Data Types
print(df.dtypes)

(45) ✓ 0.0s Python

...
sepal_length    float64
sepal_width     float64
petal_length    float64
petal_width     float64
species         str
dtype: object
```

b. Perform basic preprocessing (handling missing values or scaling).

```
b. Perform basic preprocessing (handling missing values & scaling)

# Missing Values
print(df.isnull().sum())

(46) ✓ 0.0s Python

...
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64

# Handle Missing Values
df.fillna(df.mean(numeric_only=True), inplace=True)

(47) ✓ 0.0s Python

...
   sepal_length  sepal_width  petal_length  petal_width  species
0             5.1           3.5           1.4           0.2    setosa
1             4.9           3.0           1.4           0.2    setosa
2             4.7           3.2           1.3           0.2    setosa
3             4.6           3.1           1.5           0.2    setosa
4             5.0           3.6           1.4           0.2    setosa
...          ...          ...          ...          ...      ...
145            6.7           3.0           5.2           2.3  virginica
146            6.3           2.5           5.0           1.9  virginica
147            6.5           3.0           5.2           2.0  virginica
148            6.2           3.4           5.4           2.3  virginica
149            5.9           3.0           5.1           1.8  virginica

150 rows x 5 columns

# Remove Duplicates
df.drop_duplicates(inplace=True)

(48) ✓ 0.0s Python

# Encode Categorical Data
df = pd.get_dummies(df)

(49) ✓ 0.0s Python

# Rename Columns
df.rename(columns={"sepal_length": "Sepallength"}, inplace=True)

(50) ✓ 0.0s Python

# Feature Engineering
df["Sepal_Area"] = df["Sepallength"] * df["sepal_width"]

(51) ✓ 0.0s Python

# Convert Data Types
df = df.astype(float)

(52) ✓ 0.0s Python
```

```

> print(df.columns.tolist())
[53] ✓ 0.0s Python
['Sepallength', 'sepal_width', 'petal_length', 'petal_width', 'species_setosa', 'species_versicolor', 'species_virginica', 'Sepal_Area']

# Feature Selection
X = df.drop(['species_setosa', 'species_versicolor', 'species_virginica'], axis=1)
[56] ✓ 0.0s Python

# Target (convert to labels)
y = df[['species_setosa', 'species_versicolor', 'species_virginica']].idxmax(axis=1)
[57] ✓ 0.0s Python

# Normalize
scaler = MinMaxScaler()
X_norm = scaler.fit_transform(X)
[61] ✓ 0.0s Python

# Standardize
std = StandardScaler()
X_std = std.fit_transform(X)
[62] ✓ 0.0s Python

```

c. Split the dataset into training and testing sets.

```

c. Split dataset into training and testing sets

> # Split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
[58] ✓ 0.0s Python

```

d. Train a simple classifier.

d. Train a simple classifier.

```

> # Train Model
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
[59] ✓ 0.0s Python

```

DecisionTreeClassifier ⓘ ⓘ

Parameters

	criterion	'gini'
	splitter	'best'
	max_depth	None
	min_samples_split	2
	min_samples_leaf	1
	min_weight_fraction_leaf	0.0
	max_features	None
	random_state	None
	max_leaf_nodes	None
	min_impurity_decrease	0.0
	class_weight	None
	ccp_alpha	0.0
	monotonic_cst	None

Fitted attributes

Name	Type	Value
classes_	ndarray[object](3,)	['species_setosa', 'species_versicolor', 'species_virginica']
feature_importances_	ndarray[float64](5,)	[0.01, 0., 0.59, 0.38, 0.03]
feature_names_in_	ndarray[object](5,)	['Sepallength', 'sepal_width', 'petal_length', 'petal_width', 'Sepal_Area']
max_features	int	5
n_classes	int64	np.int64(3)
n_features_in_	int	5
n_outputs	int	1
tree_	Tree	<sklearn.tree...0023D048282D0>

e. Display training and testing accuracy.

e. Display training and testing accuracy

```
# Accuracy
print("Train Accuracy:", model.score(X_train, y_train))
print("Test Accuracy:", model.score(X_test, y_test))
```

[70] ✓ 0.0s

Python

```
--- Train Accuracy: 1.0
Test Accuracy: 0.9666666666666667
```

Find one real-life business or organization dataset and perform data analysis operations.

customer_sales.csv

(customer_sales.csv) real life business or organizations dataset

```
# Import libraries
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
```

[71] ✓ 0.0s

Python

```
# Load Dataset
df = pd.read_csv("customer_sales.csv")
```

[72] ✓ 0.0s

Python

```
# Display Dataset
print(df.head())
```

[73] ✓ 0.0s

Python

```
--- Customer ID Age Gender City Product Quantity Price Payment Method \
0 CUST001 56 Male Pune Laptop 1 22886 Cash
1 CUST002 46 Female Mumbai Laptop 2 13903 Card
2 CUST003 32 Female Mumbai Mobile 1 32597 UPI
3 CUST004 25 Female Delhi Tablet 4 13621 UPI
4 CUST005 38 Male Pune Tablet 3 11466 UPI

Purchase
0 1
1 0
2 0
3 0
4 0
```

```
# Info
print(df.info())
```

[74] ✓ 0.0s

Python

```
--- <class 'pandas.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 9 columns):
# Column Non-Null Count Dtype
--- ---
0 Customer ID 100 non-null str
1 Age 100 non-null int64
2 Gender 100 non-null str
3 City 100 non-null str
4 Product 100 non-null str
5 Quantity 100 non-null int64
6 Price 100 non-null int64
7 Payment Method 100 non-null str
8 Purchase 100 non-null int64
dtypes: int64(4), str(5)
memory usage: 7.2 KB
None
```

```
# Shape
print(df.shape)
```

[75] ✓ 0.0s

Python

```
--- (100, 9)
```

```

# Columns
print(df.columns)

```

[76] ✓ 0.0s Python

```

Index(['Customer ID', 'Age', 'Gender', 'City', 'Product', 'Quantity', 'Price',
      'Payment Method', 'Purchase'],
      dtype='str')

```

```

# Summary
print(df.describe())

```

[77] ✓ 0.0s Python

```


```

	Age	Quantity	Price	Purchase
count	100.000000	100.000000	100.000000	100.000000
mean	37.910000	2.500000	24046.730000	0.460000
std	12.219454	1.184922	13998.929192	0.500908
min	18.000000	1.000000	1376.000000	0.000000
25%	26.750000	1.000000	13714.000000	0.000000
50%	38.000000	2.000000	23669.000000	0.000000
75%	46.250000	4.000000	32807.750000	1.000000
max	59.000000	4.000000	49656.000000	1.000000

```

# Data Types
print(df.dtypes)

```

[78] ✓ 0.0s Python

```

Customer ID    str
Age            int64
Gender         str
City           str
Product        str
Quantity       int64
Price          int64
Payment Method str
Purchase       int64
dtype: object

```

```

# Missing Values
print(df.isnull().sum())

```

[79] ✓ 0.0s Python

```

Customer ID    0
Age            0
Gender         0
City           0
Product        0
Quantity       0
Price          0
Payment Method 0
Purchase       0
dtype: int64

```

```

# Handle Missing Values
df = df.fillna(df.mean(numeric_only=True))

```

[81] ✓ 0.0s Python

```

# Remove Duplicates
df.drop_duplicates(inplace=True)

```

[82] ✓ 0.0s Python

```

# Encode Categorical Data
df = pd.get_dummies(df)

```

[83] ✓ 0.0s Python

```

# Rename Columns
df.rename(columns={"Customer ID": "Cust_ID"}, inplace=True)

```

[84] ✓ 0.0s Python

```

# Drop Unnecessary Columns
if "Cust_ID" in df.columns:
    df.drop("Cust_ID", axis=1, inplace=True)

```

[85] ✓ 0.0s Python

```

# Feature Engineering
df["Total_Value"] = df["Quantity"] * df["Price"]

```

[86] ✓ 0.0s Python

```

# Convert Data Types
df = df.astype(float)

```

[87] ✓ 0.0s Python

```
# Feature Selection
X = df.drop("Purchase", axis=1)
y = df["Purchase"]

# Normalize
scaler = MinMaxScaler()
X_norm = scaler.fit_transform(X)

# Standardize
std = StandardScaler()
X_std = std.fit_transform(X)

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_std = scaler.fit_transform(X)

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_std, y, test_size=0.2, random_state=42
)

# Train Model
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
```

DecisionTreeClassifier

Parameters

crit	'gini'
split	'best'
max_depth	None
min_samples_split	2
min_samples_leaf	1
min_weight_fraction_leaf	0.0
max_features	None
random_state	None
max_leaf_nodes	None
min_impurity_decrease	0.0
class_weight	None
ccp_alpha	0.0
monotonic_cst	None

Fitted attributes

Name	Type	Value
classes_	ndarray[float64](2,)	[0.,1.]
feature_importances_	ndarray[float64](117,)	[0.11,0.1 ,0.06,...,0. ,0. ,0.23]
max_features_	int	117
n_classes_	int64	np.int64(2)
n_features_in_	int	117
n_outputs_	int	1
tree_	Tree	<sklearn.tree...00230D4AF7E50>

```
# Accuracy
print("Train Accuracy:", model.score(X_train, y_train))
print("Test Accuracy:", model.score(X_test, y_test))
```

Train Accuracy: 1.0
Test Accuracy: 0.3